

Ανάλυση Δεδομένων 2025-26

4η Εργασία – Ενιαίο Explanatory Report

Δέντρα ταξινόμησης και ensemble μέθοδοι για πρόβλεψη
τύπου κρασιού

Ρακοβαλής Παύλος
ΑΕΜ: 6931

30 Απριλίου 2026

Περίληψη

Το παρόν αρχείο είναι το ενιαίο επεξηγηματικό report της 4ης εργασίας. Είναι γραμμένο με εκπαιδευτικό ύφος, σαν να απευθύνεται σε κάποιον που δεν έχει προηγούμενη εμπειρία στην Ανάλυση Δεδομένων. Για κάθε ερώτημα παρουσιάζεται πρώτα τι ζητάει η εκφώνηση, στη συνέχεια δίνεται το απαραίτητο θεωρητικό υπόβαθρο και τέλος εξηγείται βήμα προς βήμα η λύση και η ερμηνεία των αποτελεσμάτων. Σε αυτή την έκδοση έχουν ενσωματωθεί τα Ερωτήματα 1, 2, 3, 4 και 5. Τα επόμενα ερωτήματα θα προστεθούν στο ίδιο αρχείο.

Περιεχόμενα

1 Γενικό πλαίσιο της εργασίας	3
1.1 Προβλεπτικές μεταβλητές	3
1.2 Training set και test set	3
1.3 Πώς μετράμε την απόδοση	4
2 Ερώτημα 1: Δέντρο ταξινόμησης μέγιστου βάθους 2	4
2.1 Τι ζητάει το ερώτημα	4
2.2 Θεωρητικό υπόβαθρο	4
2.3 Πώς το λύσαμε	5
2.4 Αποτελέσματα	5
2.5 Ερμηνεία	6
3 Ερώτημα 2: Bagging με 200 εκτιμητές	7
3.1 Τι ζητάει το ερώτημα	7
3.2 Θεωρητικό υπόβαθρο	7
3.3 Πώς το λύσαμε	8
3.4 Αποτελέσματα	8
3.5 Σύγκριση με το Ερώτημα 1	9
3.6 Σημαντικότητα χαρακτηριστικών	10

3.7	Συμπέρασμα Ερωτήματος 2	10
4	Ερώτημα 3: Random Forest με 200 εκτιμητές και $m = p/2$	10
4.1	Τι ζητάει το ερώτημα	10
4.2	Θεωρητικό υπόβαθρο	11
4.3	Τι σημαίνει $m = p/2$	11
4.4	Πώς το λύσαμε	12
4.5	Αποτελέσματα	12
4.6	Σύγκριση με τα Ερωτήματα 1 και 2	13
4.7	Σημαντικότητα χαρακτηριστικών	14
4.8	Συμπέρασμα Ερωτήματος 3	14
5	Ερώτημα 4: Boosting με 200 επιμέρους μοντέλα	14
5.1	Τι ζητάει το ερώτημα	14
5.2	Θεωρητικό υπόβαθρο	15
5.3	Τι είναι weak learner	15
5.4	Τι σημαίνει learning rate	15
5.5	Πώς το λύσαμε	16
5.6	Αποτελέσματα	17
5.7	Confusion matrix	17
5.8	Σύγκριση με τα προηγούμενα μοντέλα	18
5.9	Σημαντικότητα χαρακτηριστικών	19
5.10	Συμπέρασμα Ερωτήματος 4	19
6	Ερώτημα 5: Test error του δέντρου συναρτήσει του βάθους	20
6.1	Τι ζητάει το ερώτημα	20
6.2	Θεωρητικό υπόβαθρο	20
6.3	Τι είναι test error	21
6.4	Πώς το λύσαμε	21
6.5	Αριθμητικά αποτελέσματα	22
6.6	Τι παρατηρούμε	22
6.7	Γιατί μετά το βάθος 6 χειροτερεύει	23
6.8	Συμπέρασμα Ερωτήματος 5	23
7	Συγκεντρωτική εικόνα μέχρι το Ερώτημα 5	24
A'	Αρχεία που χρησιμοποιούνται στο ενιαίο explanatory report	24

1 Γενικό πλαίσιο της εργασίας

Η 4η εργασία χρησιμοποιεί τα δεδομένα της 3ης εργασίας. Ο στόχος είναι να προβλέψουμε τον τύπο κρασιού, δηλαδή αν ένα κρασί είναι λευκό ή κόκκινο.

Η μεταβλητή που θέλουμε να προβλέψουμε είναι:

wine_type

και μπορεί να πάρει δύο τιμές:

Τιμή	Σημασία
red	κόκκινο κρασί
white	λευκό κρασί

Επειδή οι αλγόριθμοι μηχανικής μάθησης δουλεύουν πιο εύκολα με αριθμητικές τιμές, κωδικοποιήσαμε τη μεταβλητή στόχο ως:

red = 0 και white = 1.

1.1 Προβλεπτικές μεταβλητές

Η εκφώνηση ζητά να χρησιμοποιηθούν όλες οι διαθέσιμες μεταβλητές εκτός από τη quality. Η wine_type δεν μπαίνει φυσικά στους predictors, επειδή είναι η μεταβλητή που προσπαθούμε να προβλέψουμε.

Άρα οι προβλεπτικές μεταβλητές είναι 11:

fixed acidity	volatile acidity	citric acid
residual sugar	chlorides	free sulfur dioxide
total sulfur dioxide	density	pH
sulphates	alcohol	

1.2 Training set και test set

Τα δεδομένα χωρίστηκαν σε:

70% training set και 30% test set.

Το training set είναι το κομμάτι των δεδομένων από το οποίο μαθαίνει το μοντέλο. Το test set είναι το κομμάτι που κρατάμε στην άκρη και το χρησιμοποιούμε μόνο στο τέλος, για να δούμε αν το μοντέλο μπορεί να προβλέψει σωστά δεδομένα που δεν είχε δει κατά την εκπαίδευση.

Σύνολο	Πλήθος	Ποσοστό
Training set	3706	70%
Test set	1589	30%

Χρησιμοποιήθηκε:

random_state = 6931

ώστε ο διαχωρισμός να μπορεί να αναπαραχθεί ακριβώς. Επίσης χρησιμοποιήθηκε `stratify=y`, ώστε η αναλογία λευκών και κόκκινων κρασιών να παραμείνει παρόμοια στο training και στο test set.

1.3 Πώς μετράμε την απόδοση

Το βασικό μέτρο απόδοσης που ζητά η εργασία είναι η test accuracy:

$$\text{Test Accuracy} = \frac{\text{σωστές προβλέψεις στο test set}}{\text{σύνολο παρατηρήσεων στο test set}}.$$

Επίσης χρησιμοποιούμε το test error:

$$\text{Test Error} = 1 - \text{Test Accuracy}.$$

Αν ένα μοντέλο έχει accuracy 0.9660, αυτό σημαίνει ότι ταξινομεί σωστά περίπου το 96.60% των test παρατηρήσεων. Αντίστοιχα, test error 0.0340 σημαίνει ότι κάνει λάθος περίπου στο 3.40% των test παρατηρήσεων.

2 Ερώτημα 1: Δέντρο ταξινόμησης μέγιστου βάθους 2

2.1 Τι ζητάει το ερώτημα

Το πρώτο ερώτημα είναι:

«Να γίνει πρόβλεψη/εκτίμηση του τύπου κρασιού με δέντρο ταξινόμησης μέγιστου βάθους 2.»

Με απλά λόγια, θέλουμε να φτιάξουμε ένα μοντέλο που χρησιμοποιεί κανόνες τύπου «αν ισχύει αυτό, πήγαινε αριστερά, αλλιώς πήγαινε δεξιά» για να αποφασίσει αν ένα κρασί είναι λευκό ή κόκκινο.

Ο περιορισμός `max_depth=2` σημαίνει ότι το δέντρο πρέπει να παραμείνει πολύ απλό. Μπορεί να κάνει το πολύ δύο επίπεδα ερωτήσεων πριν καταλήξει σε τελική πρόβλεψη.

2.2 Θεωρητικό υπόβαθρο

Ένα δέντρο ταξινόμησης μοιάζει με διάγραμμα αποφάσεων. Σε κάθε κόμβο κάνει μία ερώτηση για κάποια μεταβλητή. Για παράδειγμα:

Είναι το `chlorides` μικρότερο ή ίσο από 0.05;

Ανάλογα με την απάντηση, η παρατήρηση ακολουθεί διαφορετικό κλαδί. Στο τέλος φτάνει σε ένα φύλλο, όπου το δέντρο δίνει την τελική πρόβλεψη: red ή white.

Το πλεονέκτημα των δέντρων είναι ότι είναι εύκολα ερμηνεύσιμα. Μπορούμε να δούμε τους κανόνες τους και να καταλάβουμε πώς πήραν μια απόφαση. Το μειονέκτημά τους είναι ότι, αν αφεθούν να μεγαλώσουν πάρα πολύ, μπορούν να υπερπροσαρμοστούν στα training δεδομένα. Για αυτό η εκφώνηση ζητά περιορισμό στο βάθος.

2.3 Πώς το λύσαμε

Ορίσαμε τους predictors και τη μεταβλητή στόχο:

```
features = [col for col in df.columns if col not in ("quality", "
    wine_type")]
X = df[features]
y = (df["wine_type"] == "white").astype(int)
```

Στη συνέχεια κάναμε το train/test split:

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=6931,
    stratify=y,
)
```

Τέλος εκπαιδεύσαμε το δέντρο:

```
model = DecisionTreeClassifier(max_depth=2, random_state=6931)
model.fit(X_train, y_train)
```

2.4 Αποτελέσματα

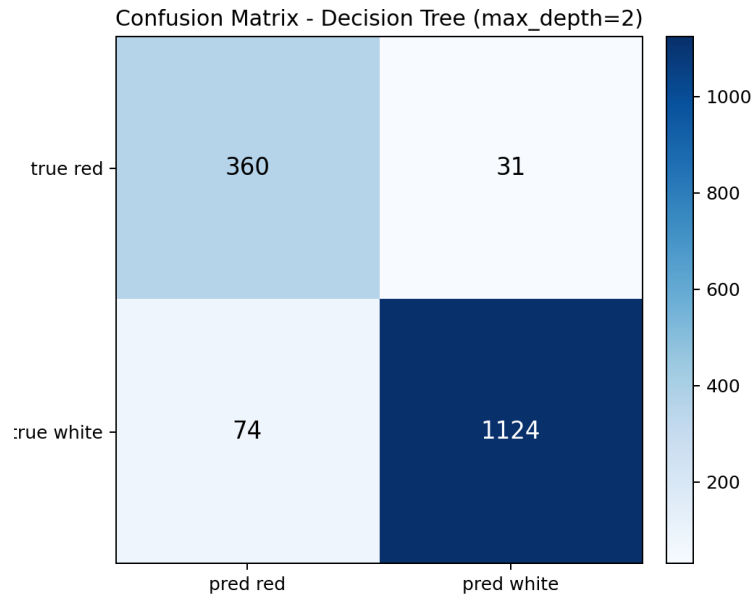
Πίνακας 1: Αποτελέσματα δέντρου ταξινόμησης με max_depth=2.

Μέτρο	Τιμή
Test accuracy	0.9339
Test error	0.0661
Σωστές προβλέψεις	1484
Λανθασμένες προβλέψεις	105

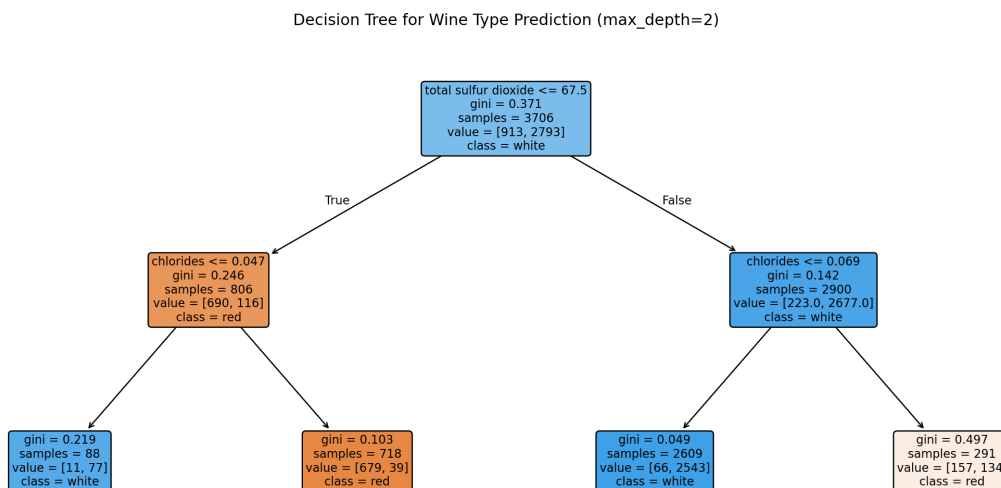
Ο confusion matrix ήταν:

Πίνακας 2: Confusion matrix για το Ερώτημα 1.

	Πρόβλεψη red	Πρόβλεψη white
Πραγματικό red	360	31
Πραγματικό white	74	1124



Σχήμα 1: Confusion matrix του δέντρου ταξινόμησης.



Σχήμα 2: Το δέντρο ταξινόμησης με μέγιστο βάθος 2.

2.5 Ερμηνεία

Το δέντρο βασίστηκε κυρίως στις μεταβλητές `total sulfur dioxide` και `chlorides`. Οι κανόνες του ήταν:

```
total sulfur dioxide <= 67.50
|-- chlorides <= 0.05 -> white
|-- chlorides > 0.05 -> red
```

```
total sulfur dioxide > 67.50
|-- chlorides <= 0.07 -> white
|-- chlorides > 0.07 -> red
```

Το μοντέλο πέτυχε accuracy 0.9339, δηλαδή ταξινομήσε σωστά περίπου το 93.39% των test παρατηρήσεων. Αυτό είναι αρκετά καλό αποτέλεσμα, ειδικά επειδή το δέντρο είναι πολύ απλό.

3 Ερώτημα 2: Bagging με 200 εκτιμητές

3.1 Τι ζητάει το ερώτημα

Το δεύτερο ερώτημα είναι:

«Να γίνει πρόβλεψη/εκτίμηση του τύπου κρασιού με τη μέθοδο Bagging, χρησιμοποιώντας 200 εκτιμητές.»

Εδώ δεν χρησιμοποιούμε ένα μόνο δέντρο. Χρησιμοποιούμε 200 δέντρα και συνδυάζουμε τις προβλέψεις τους.

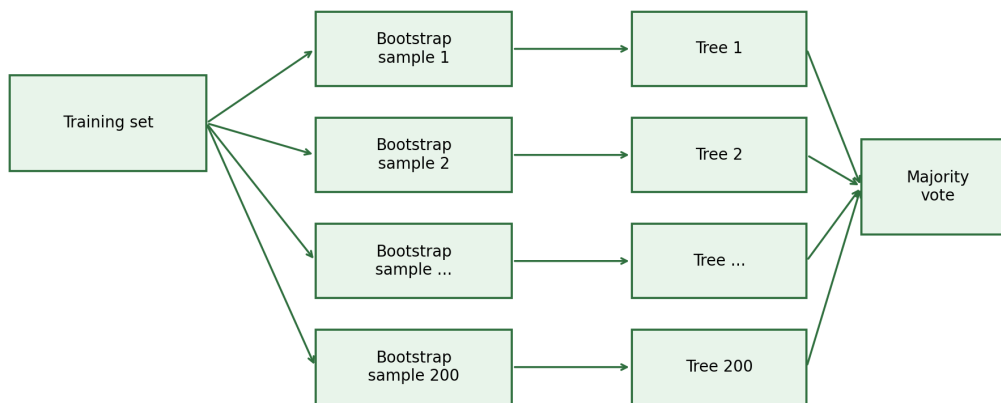
3.2 Θεωρητικό υπόβαθρο

Το Bagging προέρχεται από τη φράση Bootstrap Aggregating. Η βασική ιδέα είναι:

1. Δημιουργούμε πολλά bootstrap δείγματα από το training set.
2. Εκπαιδεύουμε ένα δέντρο σε κάθε bootstrap δείγμα.
3. Κάθε δέντρο κάνει τη δική του πρόβλεψη.
4. Η τελική πρόβλεψη προκύπτει με πλειοψηφική ψηφοφορία.

Ένα bootstrap δείγμα δημιουργείται με δειγματοληψία με επανάθεση. Αυτό σημαίνει ότι μία παρατήρηση μπορεί να εμφανιστεί περισσότερες από μία φορές στο ίδιο δείγμα, ενώ κάποια άλλη μπορεί να μην εμφανιστεί καθόλου.

Το Bagging βοηθά επειδή μειώνει την αστάθεια ενός μεμονωμένου δέντρου. Αν ένα δέντρο κάνει λάθος, η τελική απόφαση μπορεί να διορθωθεί από τα υπόλοιπα δέντρα.



Bagging: many trees are trained on different bootstrap samples, then their votes are combined.

Σχήμα 3: Η βασική ιδέα του Bagging.

3.3 Πώς το λύσαμε

Χρησιμοποιήσαμε τον ίδιο διαχωρισμό training/test με το Ερώτημα 1. Το μοντέλο ορίστηκε ως:

```

model = BaggingClassifier(
    estimator=DecisionTreeClassifier(random_state=6931),
    n_estimators=200,
    random_state=6931,
    n_jobs=-1,
)
  
```

Η παράμετρος `n_estimators=200` σημαίνει ότι το Bagging εκπαιδεύει 200 δέντρα ταξινόμησης.

3.4 Αποτελέσματα

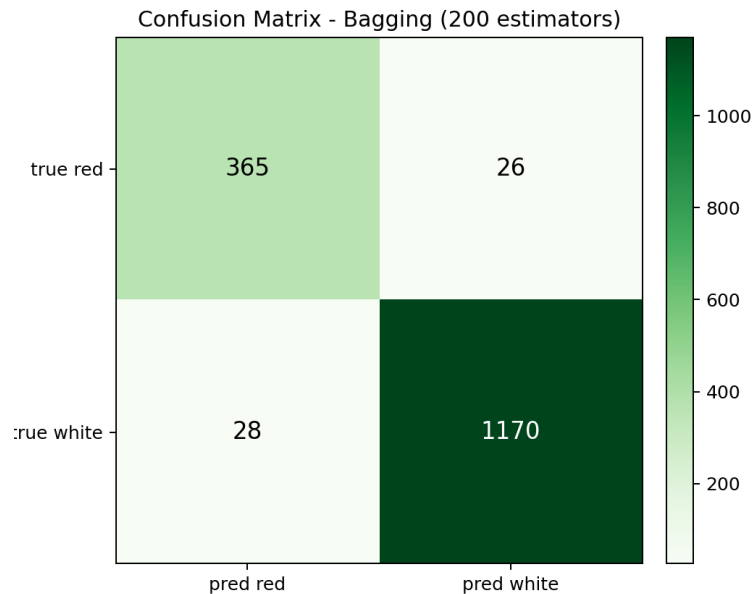
Πίνακας 3: Αποτελέσματα Bagging με 200 εκτιμητές.

Μέτρο	Τιμή
Test accuracy	0.9660
Test error	0.0340
Σωστές προβλέψεις	1535
Λανθασμένες προβλέψεις	54

Ο confusion matrix ήταν:

Πίνακας 4: Confusion matrix για το Bagging.

	Πρόβλεψη red	Πρόβλεψη white
Πραγματικό red	365	26
Πραγματικό white	28	1170



Σχήμα 4: Confusion matrix του Bagging.

3.5 Σύγκριση με το Ερώτημα 1

Πίνακας 5: Σύγκριση απλού δέντρου και Bagging.

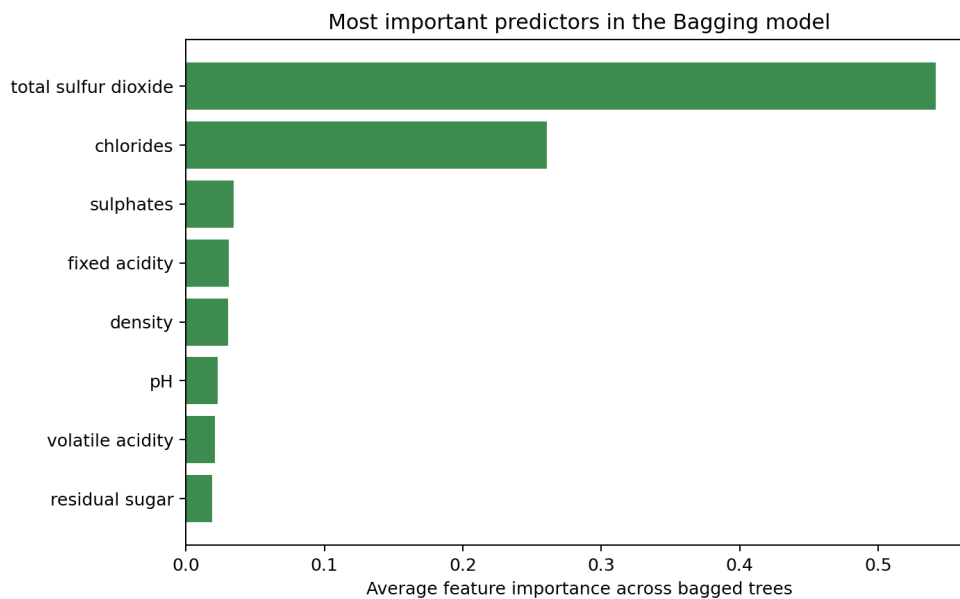
Μοντέλο	Test Accuracy	Test Error
Decision Tree (max_depth=2)	0.9339	0.0661
Bagging (200 estimators)	0.9660	0.0340

Το Bagging έκανε 54 λάθη, ενώ το απλό δέντρο έκανε 105 λάθη. Άρα το Bagging έκανε:

$$105 - 54 = 51$$

λιγότερα λάθη.

3.6 Σημαντικότητα χαρακτηριστικών



Σχήμα 5: Οι σημαντικότερες μεταβλητές στο Bagging.

Οι σημαντικότερες μεταβλητές στο Bagging ήταν κυρίως οι total sulfur dioxide και chlorides. Αυτό συμφωνεί με το απλό δέντρο του Ερωτήματος 1.

3.7 Συμπέρασμα Ερωτήματος 2

Το Bagging με 200 εκτιμητές πέτυχε test accuracy 0.9660. Η απόδοση είναι σημαντικά καλύτερη από το απλό δέντρο, επειδή το Bagging δεν βασίζεται σε ένα μόνο δέντρο αλλά σε ψηφοφορία πολλών δέντρων.

4 Ερώτημα 3: Random Forest με 200 εκτιμητές και $m = p/2$

4.1 Τι ζητάει το ερώτημα

Το τρίτο ερώτημα είναι:

«Να γίνει πρόβλεψη/εκτίμηση του τύπου κρασιού με Random Forest, χρησιμοποιώντας 200 εκτιμητές και τυχαίο πλήθος υποψήφιων χαρακτηριστικών $m = p/2$.»

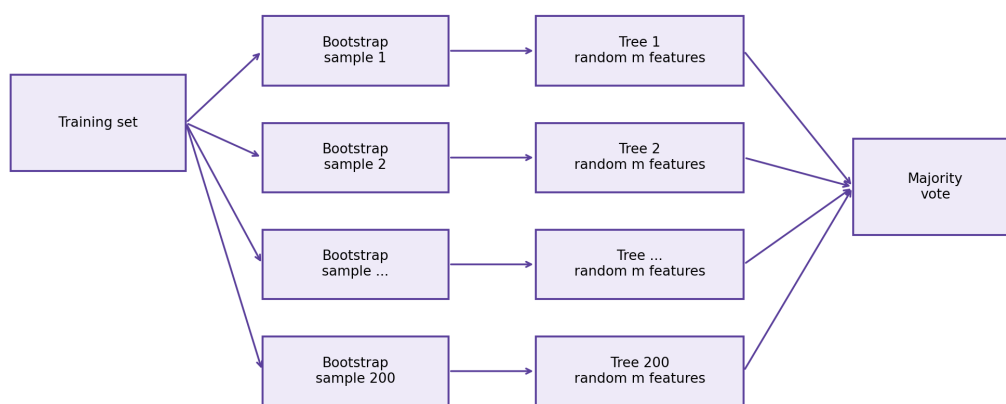
Εδώ χρησιμοποιούμε ξανά πολλά δέντρα, όπως στο Bagging. Η διαφορά είναι ότι στο Random Forest κάθε split κάθε δέντρου δεν εξετάζει όλες τις μεταβλητές, αλλά μόνο ένα τυχαίο υποσύνολο.

4.2 Θεωρητικό υπόβαθρο

Το Random Forest είναι σαν Bagging, αλλά με επιπλέον τυχαιότητα. Συγκεκριμένα:

- κάθε δέντρο εκπαιδεύεται σε bootstrap δείγμα,
- σε κάθε split εξετάζεται μόνο τυχαίο υποσύνολο χαρακτηριστικών,
- στο τέλος τα δέντρα ψηφίζουν για την τελική κλάση.

Αυτό βοηθά γιατί κάνει τα δέντρα πιο διαφορετικά μεταξύ τους. Αν όλα τα δέντρα μοιάζουν πολύ, τότε μπορεί να κάνουν παρόμοια λάθη. Αν όμως είναι πιο διαφορετικά, η πλειοψηφική ψηφοφορία γίνεται πιο χρήσιμη.



Random Forest = Bagging of trees + random subset of candidate features at each split ($m = p/2$).

Σχήμα 6: Η βασική ιδέα του Random Forest.

4.3 Τι σημαίνει $m = p/2$

Στο πρόβλημά μας οι προβλεπτικές μεταβλητές είναι:

$$p = 11.$$

Η εκφώνηση ζητά:

$$m = \frac{p}{2} = \frac{11}{2} = 5.5.$$

Επειδή το `scikit-learn` χρειάζεται ακέραιο πλήθος χαρακτηριστικών όταν δίνουμε το `max_features` ως αριθμό, χρησιμοποιήθηκε:

$$\text{max_features} = 5.$$

Άρα, σε κάθε split, το Random Forest εξέταζε τυχαία 5 από τις 11 μεταβλητές.

4.4 Πώς το λύσαμε

Ο υπολογισμός του m έγινε ως εξής:

```
features = [col for col in df.columns if col not in ("quality", "
    wine_type")]
p = len(features)
max_features = p // 2
```

Το μοντέλο ορίστηκε ως:

```
model = RandomForestClassifier(
    n_estimators=200,
    max_features=5,
    random_state=6931,
    n_jobs=-1,
)
```

4.5 Αποτελέσματα

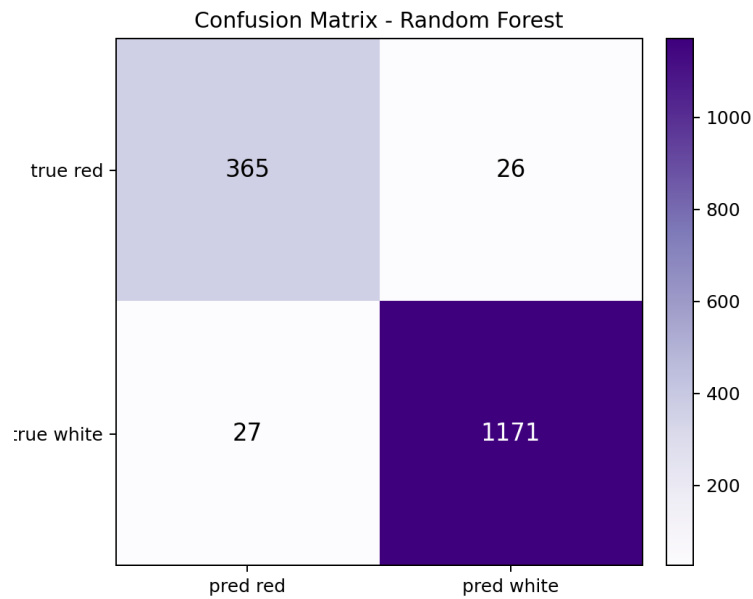
Πίνακας 6: Αποτελέσματα Random Forest.

Μέτρο	Τιμή
Πλήθος predictors p	11
$p/2$	5.5
Χρησιμοποιημένο max_features	5
Test accuracy	0.9666
Test error	0.0334
Σωστές προβλέψεις	1536
Λανθασμένες προβλέψεις	53

Ο confusion matrix ήταν:

Πίνακας 7: Confusion matrix για το Random Forest.

	Πρόβλεψη red	Πρόβλεψη white
Πραγματικό red	365	26
Πραγματικό white	27	1171



Σχήμα 7: Confusion matrix του Random Forest.

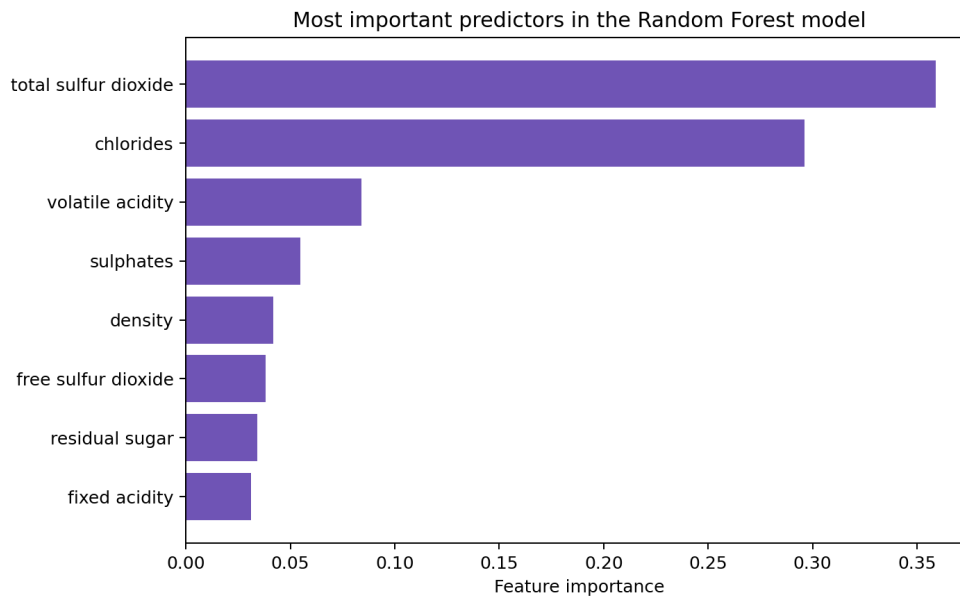
4.6 Σύγκριση με τα Ερωτήματα 1 και 2

Πίνακας 8: Σύγκριση των τριών πρώτων μοντέλων.

Μοντέλο	Test Accuracy	Test Error
Decision Tree (max_depth=2)	0.9339	0.0661
Bagging (200 estimators)	0.9660	0.0340
Random Forest (200 estimators, $m = 5$)	0.9666	0.0334

Το Random Forest έκανε 53 λάθη, δηλαδή ένα λιγότερο από το Bagging και 52 λιγότερα από το απλό δέντρο.

4.7 Σημαντικότητα χαρακτηριστικών



Σχήμα 8: Οι σημαντικότερες μεταβλητές στο Random Forest.

Οι σημαντικότερες μεταβλητές ήταν και πάλι οι total sulfur dioxide και chlorides. Αυτό είναι συνεπές με όσα είδαμε και στα δύο προηγούμενα ερωτήματα.

4.8 Συμπέρασμα Ερωτήματος 3

Το Random Forest με 200 δέντρα και max_features=5 πέτυχε test accuracy 0.9666. Η απόδοση ήταν πολύ καλύτερη από το απλό δέντρο και οριακά καλύτερη από το Bagging. Η βελτίωση σε σχέση με το Bagging είναι μικρή, αλλά το Random Forest παραμένει θεωρητικά πιο ισχυρή παραλλαγή, επειδή μειώνει τη συσχέτιση μεταξύ των δέντρων μέσω της τυχαίας επιλογής χαρακτηριστικών.

5 Ερώτημα 4: Boosting με 200 επιμέρους μοντέλα

5.1 Τι ζητάει το ερώτημα

Το τέταρτο ερώτημα είναι:

«Να γίνει πρόβλεψη/εκτίμηση του τύπου κρασιού με τη μέθοδο Boosting, χρησιμοποιώντας 200 επιμέρους μοντέλα, συντελεστή μάθησης 0.1 και μέγιστο βάθος 1.»

Με απλά λόγια, θέλουμε ξανά να προβλέψουμε αν ένα κρασί είναι λευκό ή κόκκινο, αλλά αυτή τη φορά με τη μέθοδο **Boosting**. Η βασική ιδέα του

Boosting είναι διαφορετική από το Bagging και το Random Forest. Στο Bagging και στο Random Forest τα δέντρα μπορούν να εκπαιδευτούν σχετικά ανεξάρτητα μεταξύ τους. Στο Boosting, όμως, τα μοντέλα χτίζονται **διαδοχικά**.

Δηλαδή:

- πρώτα εκπαιδεύεται ένα απλό μοντέλο,
- μετά εκπαιδεύεται ένα δεύτερο μοντέλο που προσπαθεί να διορθώσει τα λάθη του πρώτου,
- μετά ένα τρίτο που προσπαθεί να βελτιώσει ακόμη περισσότερο το σύνολο,
- και η διαδικασία συνεχίζεται μέχρι να φτάσουμε στα 200 μοντέλα.

Στο ερώτημα ζητούνται συγκεκριμένες τιμές:

```
n_estimators = 200,    learning_rate = 0.1,    max_depth = 1.
```

5.2 Θεωρητικό υπόβαθρο

Το Boosting είναι μία ensemble μέθοδος. Αυτό σημαίνει ότι δεν βασίζεται σε ένα μόνο μοντέλο, αλλά συνδυάζει πολλά μικρότερα μοντέλα.

Η σημαντική διαφορά είναι ότι στο Boosting τα μοντέλα δεν φτιάχνονται όλα ανεξάρτητα. Φτιάχνονται το ένα μετά το άλλο. Κάθε νέο μοντέλο προσπαθεί να βελτιώσει το συνολικό μοντέλο που έχει δημιουργηθεί μέχρι εκείνη τη στιγμή.

5.3 Τι είναι weak learner

Στο Boosting συχνά χρησιμοποιούμε πολύ απλά μοντέλα, τα οποία λέγονται **weak learners**. Ένας weak learner δεν είναι απαραίτητα πολύ ισχυρός μόνος του. Μπορεί να κάνει αρκετά λάθη. Όμως, όταν συνδυάσουμε πολλούς weak learners σωστά, μπορούμε να δημιουργήσουμε ένα πολύ ισχυρό μοντέλο.

Στο δικό μας ερώτημα, κάθε επιμέρους μοντέλο είναι δέντρο με:

```
max_depth = 1.
```

Ένα δέντρο βάθους 1 λέγεται συχνά decision stump. Είναι ένα πολύ απλό δέντρο, το οποίο κάνει μόνο έναν βασικό διαχωρισμό. Από μόνο του δεν έχει μεγάλη εκφραστική δύναμη, αλλά πολλά τέτοια μικρά δέντρα μαζί μπορούν να δώσουν πολύ καλή πρόβλεψη.

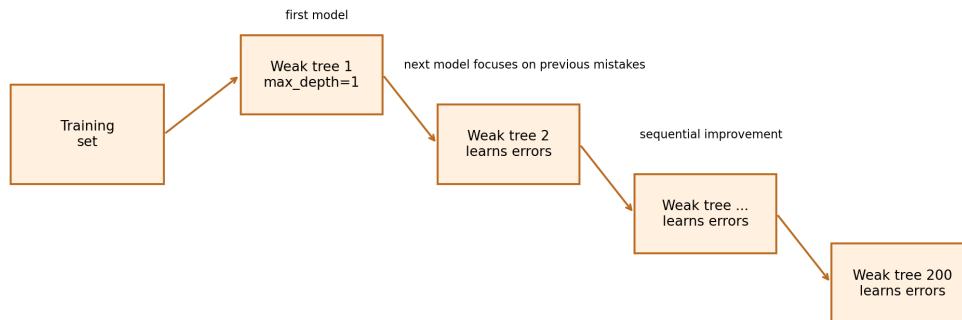
5.4 Τι σημαίνει learning rate

Η παράμετρος learning_rate ελέγχει πόσο έντονα επηρεάζει κάθε νέο μοντέλο το συνολικό ensemble.

Στο ερώτημα χρησιμοποιούμε:

`learning_rate = 0.1.`

Αυτό σημαίνει ότι κάθε νέο δέντρο προστίθεται με σχετικά μικρό βάρος. Με απλά λόγια, το μοντέλο μαθαίνει πιο αργά και πιο προσεκτικά. Ένα μικρό `learning_rate` μπορεί να βοηθήσει στη σταθερότητα της μεθόδου, επειδή αποφεύγει τις πολύ απότομες αλλαγές από ένα μόνο επιμέρους μοντέλο.



Boosting builds trees sequentially. Each new weak tree tries to improve the current ensemble.

Σχήμα 9: Η βασική ιδέα του Boosting: τα επιμέρους μοντέλα χτίζονται διαδοχικά.

5.5 Πώς το λύσαμε

Χρησιμοποιήσαμε το ίδιο dataset, τους ίδιους predictors και το ίδιο train/test split με τα προηγούμενα ερωτήματα. Η μεταβλητή στόχος ήταν πάλι η `wine_type`, με κωδικοποίηση `red=0` και `white=1`.

Το μοντέλο ορίστηκε ως:

```
model = GradientBoostingClassifier(  
    n_estimators=200,  
    learning_rate=0.1,  
    max_depth=1,  
    random_state=6931,  
)
```

Ας δούμε τι σημαίνει κάθε παράμετρος:

- `GradientBoostingClassifier`: δηλώνει ότι χρησιμοποιούμε Boosting για ταξινόμηση.
- `n_estimators=200`: το μοντέλο θα αποτελείται από 200 επιμέρους δέντρα.
- `learning_rate=0.1`: κάθε νέο δέντρο επηρεάζει το συνολικό μοντέλο με σχετικά μικρό βήμα.
- `max_depth=1`: κάθε επιμέρους δέντρο είναι πολύ απλό.

- `random_state=6931`: τα αποτελέσματα είναι αναπαραγώγιμα.

Μετά την εκπαίδευση, έγιναν προβλέψεις στο test set:

```
y_pred = model.predict(X_test)
```

και υπολογίστηκαν:

```
accuracy = accuracy_score(y_test, y_pred)
test_error = 1 - accuracy
```

5.6 Αποτελέσματα

Πίνακας 9: Αποτελέσματα Boosting.

Μέτρο	Τιμή
n_estimators	200
learning_rate	0.1
max_depth	1
Test accuracy	0.9679
Test error	0.0321
Σωστές προβλέψεις	1538
Λανθασμένες προβλέψεις	51

Το Boosting πέτυχε:

Test accuracy = 0.9679

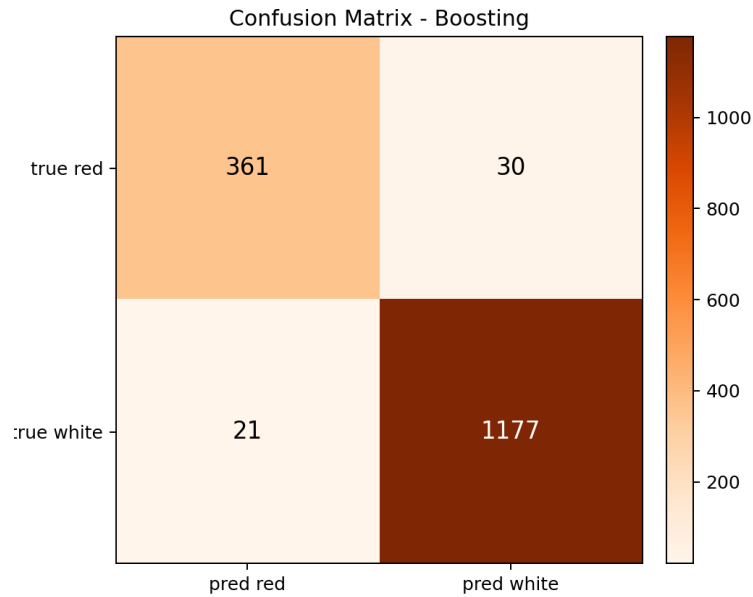
δηλαδή ταξινομήσε σωστά περίπου το 96.79% των test παρατηρήσεων.

5.7 Confusion matrix

Ο confusion matrix ήταν:

Πίνακας 10: Confusion matrix για το Boosting.

	Πρόβλεψη red	Πρόβλεψη white
Πραγματικό red	361	30
Πραγματικό white	21	1177



Σχήμα 10: Confusion matrix του Boosting.

Ο πίνακας δείχνει ότι:

- 361 κόκκινα κρασιά ταξινομήθηκαν σωστά ως κόκκινα,
- 30 κόκκινα κρασιά ταξινομήθηκαν λάθος ως λευκά,
- 21 λευκά κρασιά ταξινομήθηκαν λάθος ως κόκκινα,
- 1177 λευκά κρασιά ταξινομήθηκαν σωστά ως λευκά.

Άρα οι σωστές προβλέψεις είναι:

$$361 + 1177 = 1538.$$

Τα συνολικά λάθη είναι:

$$30 + 21 = 51.$$

5.8 Σύγκριση με τα προηγούμενα μοντέλα

Πίνακας 11: Σύγκριση των τεσσάρων πρώτων μοντέλων.

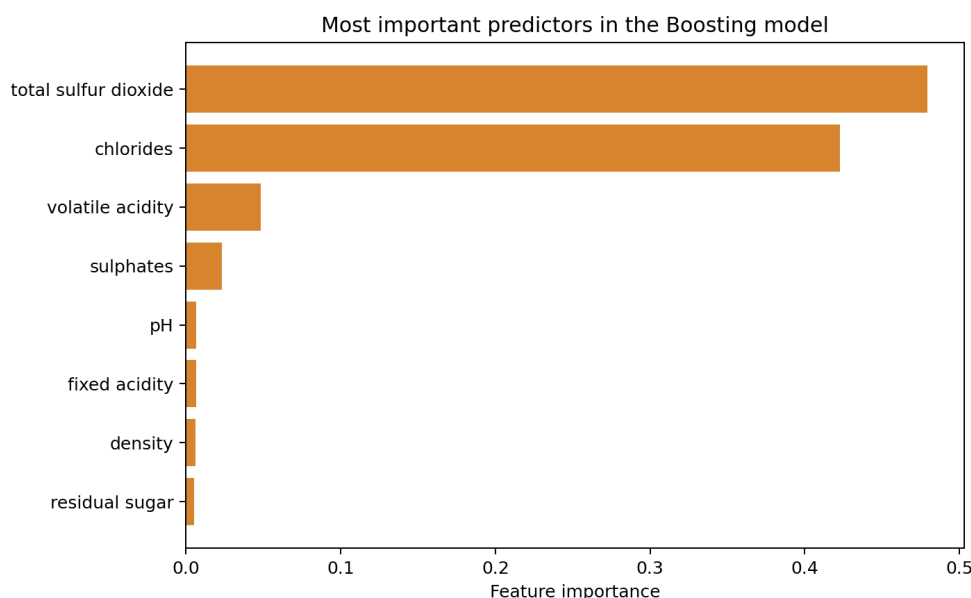
Μοντέλο	Test Accuracy	Test Error
Decision Tree (max_depth=2)	0.9339	0.0661
Bagging (200 estimators)	0.9660	0.0340
Random Forest (200 estimators, $m = 5$)	0.9666	0.0334
Boosting (200 estimators)	0.9679	0.0321

Μέχρι τώρα, το Boosting έχει την καλύτερη test accuracy. Η διαφορά από το Random Forest είναι μικρή, αλλά το Boosting έκανε τα λιγότερα συνολικά λάθη:

Μοντέλο	Λάθη	Σωστές προβλέψεις
Decision Tree	105	1484
Bagging	54	1535
Random Forest	53	1536
Boosting	51	1538

5.9 Σημαντικότητα χαρακτηριστικών

Το Boosting δίνει επίσης μία ένδειξη για το ποιες μεταβλητές βοήθησαν περισσότερο στην ταξινόμηση.



Σχήμα 11: Οι σημαντικότερες μεταβλητές στο Boosting.

Οι δύο σημαντικότερες μεταβλητές ήταν και εδώ οι:

- total sulfur dioxide
- chlorides

Αυτό επιβεβαιώνει για ακόμη μία φορά ότι αυτές οι μεταβλητές είναι πολύ χρήσιμες για τη διάκριση λευκού και κόκκινου κρασιού.

5.10 Συμπέρασμα Ερωτήματος 4

Το Boosting με 200 επιμέρους μοντέλα, `learning_rate=0.1` και `max_depth=1` πέτυχε test accuracy 0.9679. Είναι το καλύτερο αποτέλεσμα μέχρι στιγμής ανάμεσα στα αρχικά μοντέλα των Ερωτημάτων 1-4.

Η μέθοδος πέτυχε υψηλή απόδοση παρότι κάθε επιμέρους δέντρο ήταν πολύ απλό. Αυτό δείχνει τη δύναμη του Boosting: πολλά απλά μοντέλα, όταν συνδυαστούν διαδοχικά και σωστά, μπορούν να δημιουργήσουν έναν πολύ ισχυρό ταξινομητή.

6 Ερώτημα 5: Test error του δέντρου συναρτήσεως του βάθους

6.1 Τι ζητάει το ερώτημα

Το πέμπτο ερώτημα είναι:

«Να παρασταθεί γραφικά το test error της μεθόδου του ερωτήματος 1 συναρτήσεως του βάθους του δέντρου. Τι παρατηρείτε; Σχολιάστε.»

Με απλά λόγια, μέχρι τώρα στο Ερώτημα 1 χρησιμοποιήσαμε ένα δέντρο με `max_depth=2`. Τώρα η εκφώνηση μάς ζητά να μην κρατήσουμε σταθερό το βάθος, αλλά να δοκιμάσουμε πολλά διαφορετικά βάθη και να δούμε πώς αλλάζει το test error.

Το ερώτημα θέλει να απαντήσουμε σε κάτι πολύ σημαντικό:

Αν κάνουμε το δέντρο πιο βαθύ, γίνεται πάντα καλύτερο;

Η απάντηση, όπως θα δούμε, είναι όχι. Μέχρι ένα σημείο το μεγαλύτερο βάθος βοηθά. Μετά από ένα σημείο, όμως, το δέντρο γίνεται υπερβολικά σύνθετο και μπορεί να αρχίσει να γενικεύει χειρότερα.

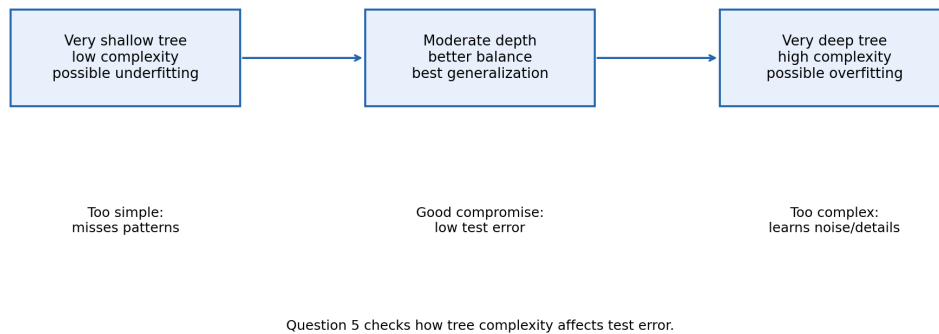
6.2 Θεωρητικό υπόβαθρο

Το βάθος ενός δέντρου καθορίζει πόσο σύνθετο μπορεί να γίνει το μοντέλο.

- Ένα πολύ ρηχό δέντρο έχει λίγους κανόνες και είναι πολύ απλό.
- Ένα πολύ βαθύ δέντρο έχει πολλούς κανόνες και μπορεί να ξεχωρίσει λεπτομέρειες στα δεδομένα.

Αν το δέντρο είναι υπερβολικά απλό, μπορεί να μην καταλαβαίνει αρκετά καλά τη σχέση ανάμεσα στα χαρακτηριστικά και στον τύπο κρασιού. Αυτό λέγεται **underfitting**.

Αν το δέντρο είναι υπερβολικά βαθύ, μπορεί να αρχίσει να μαθαίνει όχι μόνο το γενικό μοτίβο, αλλά και τυχαίες λεπτομέρειες του training set. Αυτό λέγεται **overfitting**.



Σχήμα 12: Η βασική ιδέα του Ερωτήματος 5: πολύ μικρό, μέτριο και πολύ μεγάλο βάθος.

6.3 Τι είναι test error

Το test error είναι:

$$\text{Test Error} = 1 - \text{Test Accuracy}.$$

Αν ένα μοντέλο έχει test accuracy 0.9339, τότε:

$$\text{Test Error} = 1 - 0.9339 = 0.0661.$$

Το test error δείχνει το ποσοστό των test παρατηρήσεων που ταξινομήθηκαν λάθος. Όσο μικρότερο είναι, τόσο καλύτερο είναι το μοντέλο.

6.4 Πώς το λύσαμε

Χρησιμοποιήσαμε το ίδιο dataset και το ίδιο train/test split με τα προηγούμενα ερωτήματα. Στη συνέχεια εκπαιδεύσαμε πολλά δέντρα ταξινόμησης, αλλάζοντας την τιμή του `max_depth`.

Δοκιμάσαμε βάθη:

$$\text{max_depth} = 1, 2, 3, \dots, 20.$$

Για κάθε βάθος εκπαιδεύτηκε ξεχωριστό δέντρο και υπολογίστηκε το test error:

```

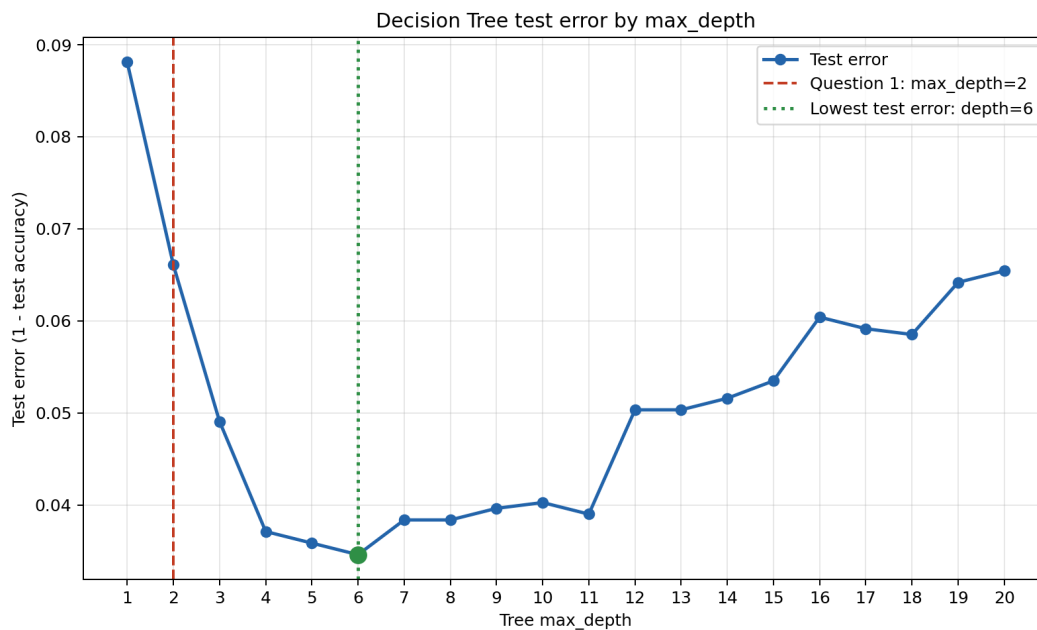
for depth in range(1, 21):
    model = DecisionTreeClassifier(max_depth=depth, random_state
    =6931)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)
    test_error = 1 - accuracy
  
```

Έτσι πήραμε έναν πίνακα που δείχνει πώς αλλάζει το test error ανάλογα με το βάθος του δέντρου.

6.5 Αριθμητικά αποτελέσματα

Πίνακας 12: Test accuracy και test error για διαφορετικά βάθη δέντρου.

Depth	Accuracy	Error	Depth	Accuracy	Error
1	0.9119	0.0881	11	0.9610	0.0390
2	0.9339	0.0661	12	0.9497	0.0503
3	0.9509	0.0491	13	0.9497	0.0503
4	0.9629	0.0371	14	0.9484	0.0516
5	0.9641	0.0359	15	0.9465	0.0535
6	0.9654	0.0346	16	0.9396	0.0604
7	0.9616	0.0384	17	0.9408	0.0592
8	0.9616	0.0384	18	0.9415	0.0585
9	0.9604	0.0396	19	0.9358	0.0642
10	0.9597	0.0403	20	0.9346	0.0654



Σχήμα 13: Test error του δέντρου ταξινόμησης συναρτήσει του max_depth.

6.6 Τι παρατηρούμε

Στα πολύ μικρά βάθη, το test error είναι σχετικά υψηλό. Για παράδειγμα:

- για max_depth=1, το test error είναι 0.0881,
- για max_depth=2, το test error είναι 0.0661.

Αυτό σημαίνει ότι ένα πολύ απλό δέντρο δεν αρκεί για να αποτυπώσει πλήρως τη δομή των δεδομένων. Έχει λίγους κανόνες και άρα δεν μπορεί να διαχωρίσει τέλεια τα λευκά από τα κόκκινα κρασιά.

Καθώς αυξάνεται το βάθος, το test error μειώνεται. Η καλύτερη τιμή εμφανίζεται στο:

$$\text{max_depth} = 6.$$

Για αυτό το βάθος έχουμε:

$$\text{Test Accuracy} = 0.9654, \quad \text{Test Error} = 0.0346.$$

Άρα το δέντρο βάθους 6 είναι πολύ καλύτερο από το αρχικό δέντρο βάθους 2. Συγκεκριμένα, το test error μειώνεται από 0.0661 σε 0.0346.

6.7 Γιατί μετά το βάθος 6 χειροτερεύει

Μετά το βάθος 6, το test error δεν συνεχίζει να μειώνεται. Αντίθετα, αρχίζει γενικά να αυξάνεται. Για παράδειγμα:

- στο βάθος 10, το test error είναι 0.0403,
- στο βάθος 15, το test error είναι 0.0535,
- στο βάθος 20, το test error είναι 0.0654.

Αυτό είναι ένδειξη overfitting. Το βαθύτερο δέντρο μαθαίνει όλο και πιο λεπτομερείς κανόνες από το training set. Αυτοί οι κανόνες μπορεί να ταιριάζουν καλά στα training δεδομένα, αλλά να μην είναι τόσο χρήσιμοι για νέα δεδομένα.

Με άλλα λόγια:

Το πιο σύνθετο μοντέλο δεν είναι πάντα το καλύτερο μοντέλο.

6.8 Συμπέρασμα Ερωτήματος 5

Το test error μειώνεται καθώς το βάθος αυξάνεται από 1 έως 6, επειδή το δέντρο αποκτά περισσότερη ευελιξία και μπορεί να περιγράψει καλύτερα τη δομή των δεδομένων. Η καλύτερη τιμή εμφανίζεται στο $\text{max_depth}=6$, με test accuracy 0.9654 και test error 0.0346.

Για μεγαλύτερα βάθη, το test error αρχίζει να αυξάνεται. Αυτό δείχνει ότι το δέντρο γίνεται υπερβολικά σύνθετο και πιθανόν υπερπροσαρμόζεται στα training δεδομένα. Επομένως, ένα μέτριο βάθος δέντρου προσφέρει την καλύτερη ισορροπία ανάμεσα σε απλότητα και προβλεπτική ικανότητα.

7 Συγκεντρωτική εικόνα μέχρι το Ερώτημα 5

Πίνακας 13: Συγκεντρωτικός πίνακας αποτελεσμάτων μέχρι το Ερώτημα 5.

Ερώτημα	Μέθοδος	Accuracy	Error
1	Decision Tree	0.9339	0.0661
2	Bagging	0.9660	0.0340
3	Random Forest	0.9666	0.0334
4	Boosting	0.9679	0.0321
5	Best Tree Depth (6)	0.9654	0.0346

Μέχρι στιγμής, το Boosting έχει την καλύτερη ακρίβεια ανάμεσα στα αρχικά μοντέλα των Ερωτημάτων 1-4. Το Ερώτημα 5 δείχνει όμως ότι το αρχικό δέντρο βάθους 2 δεν ήταν το καλύτερο δυνατό απλό δέντρο. Αν αυξήσουμε το βάθος μέχρι το 6, το δέντρο φτάνει σε test accuracy 0.9654, δηλαδή πλησιάζει πολύ τις ensemble μεθόδους.

Α' Αρχεία που χρησιμοποιούνται στο ενιαίο explanatory report

Τα βασικά βοηθητικά αρχεία που έχουν δημιουργηθεί μέχρι τώρα είναι:

- q1_explanatory_decision_tree.py
- q2_explanatory_bagging.py
- q3_explanatory_random_forest.py
- q4_explanatory_boosting.py
- q5_explanatory_tree_depth.py
- q1_confusion_matrix.png
- q1_decision_tree_plot.png
- q2_confusion_matrix.png
- q2_bagging_concept.png
- q2_bagging_feature_importance.png
- q3_confusion_matrix.png
- q3_random_forest_concept.png
- q3_random_forest_feature_importance.png
- q4_confusion_matrix.png
- q4_boosting_concept.png
- q4_boosting_feature_importance.png

- q5_tree_depth_concept.png
- q5_tree_depth_error.png